Chapter 14

SonarQube: Continuous Code Quality & SAST

Every commit, automatically scanned for bugs, vulnerabilities, code smells, and coverage gaps — before it ever reaches production.

Time estimate: 3–4 hours

Deployed on sec-appsec-01 (10.0.2.15)

What You'll Build in This Chapter

Deploy SonarQube on sec-appsec-01 for Static Application Security Testing (SAST). Every commit is automatically scanned for bugs, vulnerabilities, code smells, and coverage metrics.

- SonarQube scans code in the CI/CD pipeline
- Bugs and vulnerabilities detected automatically
- Code coverage metrics are tracked
- Merge requests blocked if quality fails
- Audit trail of every scan

The Analogy

SonarQube is like a building inspector:

- Inspects the code — walks through, checks for issues
- Finds problems early — before deployment
- Prevents disasters — security, stability, maintainability
- Tracks progress — quality trends over time

Security starts with code quality — most breaches trace back to a bug that was always there.

Four Categories of Issues SonarQube Finds

Type	Example	Risk
Bugs	Null pointer exception	Runtime crash
Vulnerabilities	SQL injection	Security breach
Code Smells	Duplicated code	Maintenance issues
Coverage Gaps	Untested code path	Unknown risks



These four categories map directly to SonarQube's dashboard tiles — the first thing you see after every scan.



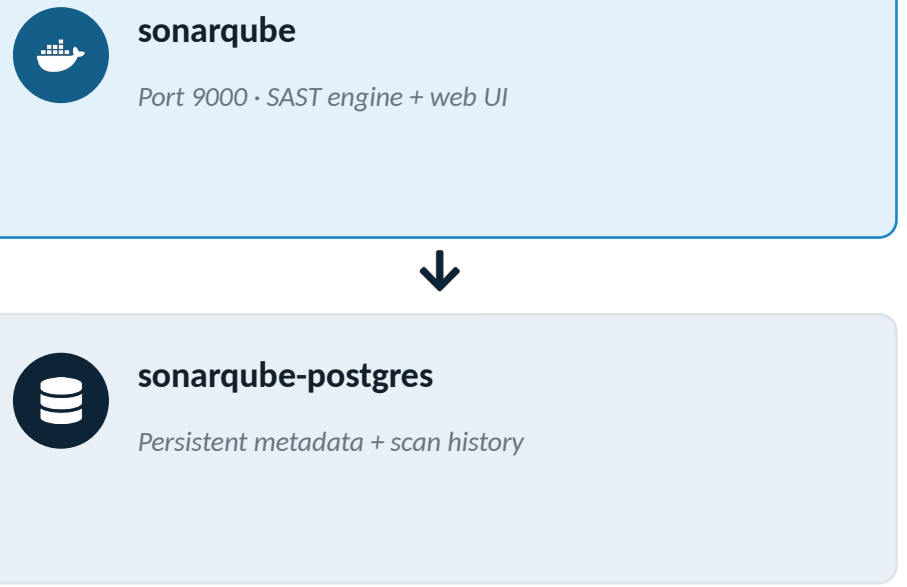
Analogy — *SonarQube is like a building inspector: it inspects the code, finds problems early, prevents disasters, and tracks progress over time.*

Docker Compose — SonarQube + PostgreSQL

```
vm# mkdir -p /opt/sonarqube/{data,extensions,\
    logs,postgres_data}
vm# cd /opt/sonarqube

vm# cat > docker-compose.yml << 'EOF'
  services:
    sonarqube:
      image: sonarqube:lts
      ports: ["9000:9000"]
    postgres:
      image: postgres:14
  EOF

vm# docker-compose up -d
```



Startup takes 1–2 minutes. Follow with docker-compose logs -f sonarqube and wait for "SonarQube is operational" before opening the web UI — hitting it too early just shows a blank page.

docker-compose.yml — Database & LDAP Wiring

```
environment:
  SONAR_JDBC_URL: jdbc:postgresql://
    postgres:5432/sonarqube
  SONAR_JDBC_USERNAME: sonarqube
  SONAR_HOST_URL: https://sonarqube.
    devsecops.com

  # LDAP authentication
  SONAR_LDAP_URL: ldap://sec-idm-01:389
  SONAR_LDAP_BINDDN: uid=service-account,
    ou=people,dc=devsecops,dc=com
  SONAR_LDAP_USER_BASEDN: ou=people,
    dc=devsecops,dc=com
  SONAR_LDAP_GROUP_BASEDN: ou=groups,
    dc=devsecops,dc=com
```

JDBC connection

Points SonarQube at the Postgres container for all persistent state

LDAP bind

Authenticates using sec-idm-01 — the same directory from Chapter 07

User & group base DN

Maps LDAP OUs so SonarQube roles follow existing group membership

Four Setup Steps After First Login

1. Log in	`https://10.0.2.15:9000`	*Default credentials: admin / admin*
2. Change password	`Administration → Users → admin`	*Rotate the default admin password immediately*
3. Generate CI token	`Profile → Security → Tokens`	*Name: gitlab-ci · Type: Global analysis token*
4. Create project	`Projects → Create Project`	*Name & Key: myapp*

Wire SonarQube Into the CI/CD Pipeline

```
# .gitlab-ci.yml
sonarqube:
  stage: scan
  image: sonarsource/sonar-scanner-cli:latest
  variables:
    SONAR_HOST_URL: "https://sonarqube.devsecops.com"
    SONAR_TOKEN: $SONARQUBE_TOKEN
  script:
    - sonar-scanner \
        -Dsonar.projectKey=myapp \
        -Dsonar.sources=. \
        -Dsonar.host.url=$SONAR_HOST_URL \
        -Dsonar.login=$SONAR_TOKEN \
        -Dsonar.qualitygate.wait=true
  allow_failure: true
  only:
    - merge_requests
    - main
    - master
```



Set the token

Project → Settings → CI/CD → Variables. Add SONARQUBE_TOKEN using the token generated in the previous step.

`allow_failure: true` means the pipeline still shows results even if the scan fails — useful while tuning quality gates before enforcing them.

Quality Gates Decide What Blocks a Merge

Administration → Configuration → Quality Gates · Gate name: DevSecOps Lab

FAIL

Coverage < 80%

WARN

Duplicated Lines > 3%

FAIL

Code Smells > 100

FAIL

Bugs > 0

FAIL

Vulnerabilities > 0

WARN

Security Hotspots > 0

Two Real Examples SonarQube Catches

app.py — vulnerable code

```
def get_user(user_id):

    query = f"SELECT * FROM users
      WHERE id = {user_id}"

    return db.execute(query)
```



SQL Injection — Security Hotspot

Severity: CRITICAL

Message: "User input is used directly in SQL query"

Fix: Use parameterized queries

app.py — vulnerable code

```
def process_data(data):

    return data.strip()
```



Potential Null Pointer Exception

Severity: MAJOR

Message: "data might be None"

Fix: Guard with a None check before .strip()

Feed Test Coverage Into the Scan



Two pipeline stages, one report

`test` runs pytest and writes coverage.xml as a build artifact; `sonarqube` consumes that same file so coverage percentage shows up right next to bugs and vulnerabilities on the dashboard.

Flagged for Human Review, Not Auto-Failed

Hotspots aren't automatically bugs — they're code patterns that need a reviewer's judgment.

Hard-coded Credentials

SECURITY HOTSPOT

```
db_password = "MySecretPassword123"
```

Message: *"Hard-coded credential detected"*

Fix: Use Vault or environment variables

Insecure Deserialization

SECURITY HOTSPOT

```
data = pickle.loads(user_input)
```

Message: *"Deserialization from untrusted source is dangerous"*

Fix: Validate input or use a safe serialization format

What the Project Dashboard Shows

Quality Gate

Passing / Failing

Code Smells

Maintainability issues

Bugs

Correctness issues

Vulnerabilities

Security issues

Coverage

% of code tested

Duplications

Duplicated code blocks

Historical Trending — Projects → myapp → Activity tracks quality trends per commit over time.

Export Reports — Administration → System → Background Tasks. Export as PDF/CSV for compliance records.

Custom Rules for Organization-Specific Standards

Administration → Rules → Custom Rules

```
Name:  "No console.log in production code"
Language:  Python
Pattern:  console\.log\(
Message:  "Remove debug statements before commit"
Severity:  MINOR
```

Beyond the defaults

SonarQube ships with hundreds of built-in rules, but every team has its own conventions. Custom rules let you enforce those conventions with the same severity and quality-gate mechanics as any built-in check.

Three Problems You'll Likely Hit

Cannot connect to SonarQube

Scanner can't reach the host

Verify with docker-compose ps + logs; check sonar-scanner -Dsonar.host.url output for the actual error

Quality gate failed, but want to see results

Pipeline blocks before you can review

Set allow_failure: true in .gitlab-ci.yml, then review results manually in the merge request

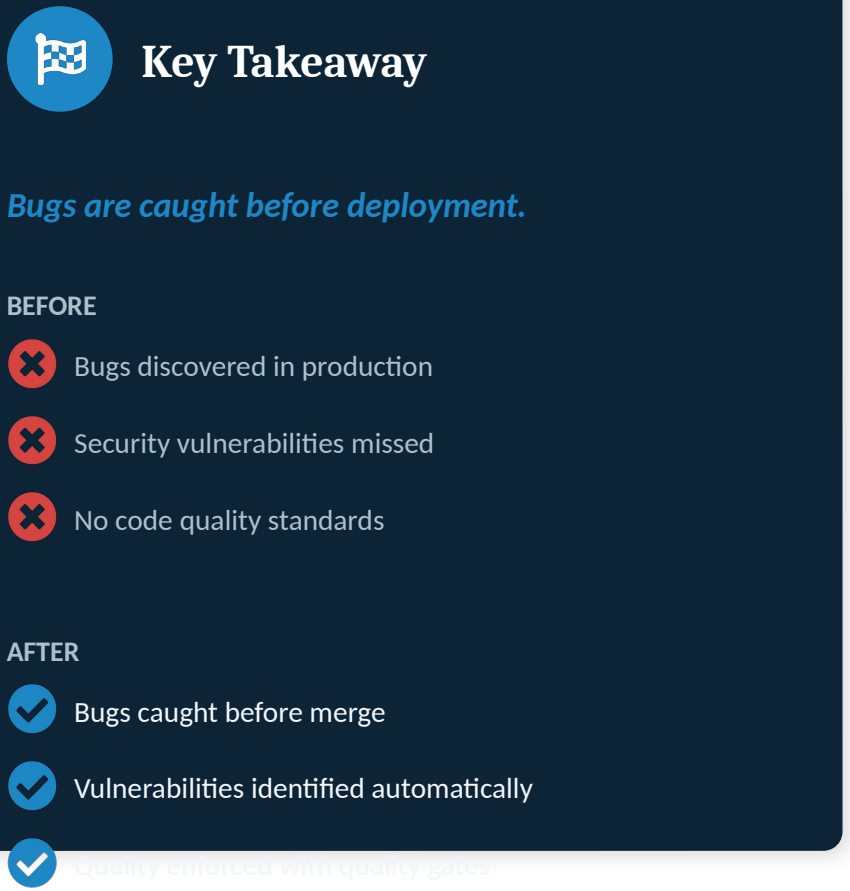
LDAP authentication not working

Bind credentials or DN mismatch

Test directly with ldapsearch against sec-idm-01; confirm the user exists in Postgres via psql

Chapter Summary — What We Built

What We Built	Purpose
SonarQube server	Static code analysis (SAST)
LDAP authentication	Users log in with LDAP credentials
Quality gates	Block merge if quality fails
GitLab integration	Scan every merge request
Security hotspots	Identify potential vulnerabilities
Code coverage	Track test coverage
Audit trail	Every scan logged



PHASE 4 CONTINUES

Code Quality Is Enforced.
Next: Securing the Container Supply Chain.

Chapter 15 → **Harbor: Container Registry & Vulnerability Scanning**

- Private container registry for all lab images
- Every image scanned for known CVEs on push
- Block deployment of critical-vulnerability images
- Integrates with GitLab CI/CD build stage
- Full audit trail of every image pushed and pulled